# LMA Project Report:

## Tokenization, Transformers, and Reasoning in Hindi and Nepali

**Anurag Kaushal**
Roll No: 2025202013
Course: Language Model Architecture (LMA)
Individual Submission

September 17, 2026

**Abstract**

In this project for the Language Model Architecture (LMA) course, I built and evaluated language models for Hindi and Nepali from scratch across three phases. In **Phase 1**, I collected over 500 million tokens of text for each language and trained custom Byte-Pair Encoding (BPE) tokenizers, choosing a vocabulary size of 16,000 for both languages. In **Phase 2**, I implemented a 24M-parameter decoder-only Transformer in PyTorch. I evaluated both models and found that while Nepali had a higher perplexity (40.70 vs Hindi's 29.81), it actually had a better (lower) Bits-Per-Byte score (0.4314 vs 0.4947). In **Phase 3**, I fine-tuned the models to do comparative reasoning (e.g., "who is taller?"). The models got really good at in-domain tasks (around 95% accuracy for Hindi and 83% for Nepali) and learned a specific attention circuit (L9H4) to copy answers. However, they both completely failed to generalize to longer chains (4-hops), showing they learned a positional trick rather than true reasoning.

---

# 1 Introduction

Working with languages like Hindi and Nepali can be tricky because their Devanagari script uses complex Unicode representations where a single character can take up to three UTF-8 bytes. Standard tokenizers mostly trained on English don't work well for this. Also, it's interesting to compare how model training differs between a higher-resource language (Hindi) and a lower-resource one (Nepali) when they use the same script.

To understand these issues, I broke down this project into three main phases:

1. **Phase 1 (Tokenization):** Collecting the data, cleaning it up, and training my own BPE tokenizers to find a good vocabulary size.
2. **Phase 2 (Transformer Pre-training):** Writing a Transformer decoder from scratch in PyTorch, tuning hyperparameters like learning rate, and comparing the models using Bits-Per-Byte (BPB) to see which one is actually better.
3. **Phase 3 (Reasoning):** Fine-tuning these models on reasoning tasks (like figuring out who is the tallest in a group) to see if small models can actually reason or if they just memorize patterns.

# 2 Phase 1: Data and Tokenization

## 2.1 Corpus and Preprocessing

For this project, I scraped and downloaded raw text datasets for Hindi and Nepali. I used the Yi pipeline approach to clean the data, which covered language filtering (Devanagari script ratio + a Hindi/Nepali function-word discriminator to stop cross-language contamination), text-metric filters (length, symbol ratio, short-line ratio), and repetition removal. I also applied NFC Unicode normalisation first, because the same Devanagari character can be encoded two different ways and the tokenizer would otherwise learn two separate vocabularies for identical text. For exact deduplication

I used SHA-256 hashing; for near-deduplication I used MinHash with $k = 100$ hash functions, $b = 20$ bands, $r = 5$ rows, giving a Jaccard similarity threshold of $s^* = (1/b)^{1/r} \approx 0.549$. I held out test data **before** deduplication so the dedup can catch near-duplicates of val/test docs in train.

Table 1 shows the size of the datasets after all the cleaning. The splits are roughly 98% train / 1% val / 1% test, assigned by a hash of the document content (seed 1337) so they are reproducible on any machine.

Table 1: Corpus Statistics for Hindi and Nepali

| Language | Documents | Tokens (own tokenizer) | Manual Data Share | Near-duplicates removed |
|---|---|---|---|---|
| Hindi (H) | 1,202,072 | 582,711,232 | 19.29% | 15,503 (1.27%) |
| Nepali (L) | 2,180,424 | 903,937,802 | 22.40% | 120,689 (5.24%) |

I want to point out a few honest things here: - Hindi's manual share is slightly below 20% (19.29%). This is because I over-collected downloaded text as a buffer, and I didn't want to throw away perfectly good data just to artificially bump up a percentage. - Nepali has way more near-duplicates (5.24% vs 1.27%). This is because Nepali news sites tend to copy-paste wire stories from each other a lot more than Hindi sites do. The median Jaccard similarity of removed pairs was 0.847 for Nepali and 0.803 for Hindi, confirming these are genuine duplicates.

### 2.2 BPE Sweep Results

I trained BPE tokenizers from scratch. I tested vocabulary sizes of 8k, 16k, 32k, and 48k. I looked at fertility (how many tokens per word) and bytes per token.

Table 2: BPE Vocabulary Sweep Results for Hindi and Nepali

| | **Hindi** | | **Nepali** | |
|---|---|---|---|---|
| **Vocab Size ($V$)** | **Fertility** | **Chars/Token** | **Fertility** | **Chars/Token** |
| 8,000 | 1.526 | 3.501 | 1.529 | 4.235 |
| **16,000** | **1.386** | **3.854** | **1.360** | **4.760** |
| 32,000 | 1.296 | 4.123 | 1.252 | 5.171 |
| 48,000 | 1.258 | 4.247 | 1.208 | 5.359 |

As shown in Table 2 and Figures 1 and 2, I picked $16,000$ for **both** languages. I did the math: at a 25M parameter budget, going to a 48k vocabulary would spend 74% of my model just on the embedding matrix for only a tiny 9% improvement in fertility. 16k is the sweet spot. Also, interesting finding: Nepali actually encodes text more efficiently (more characters per token) than Hindi at the same vocab size.

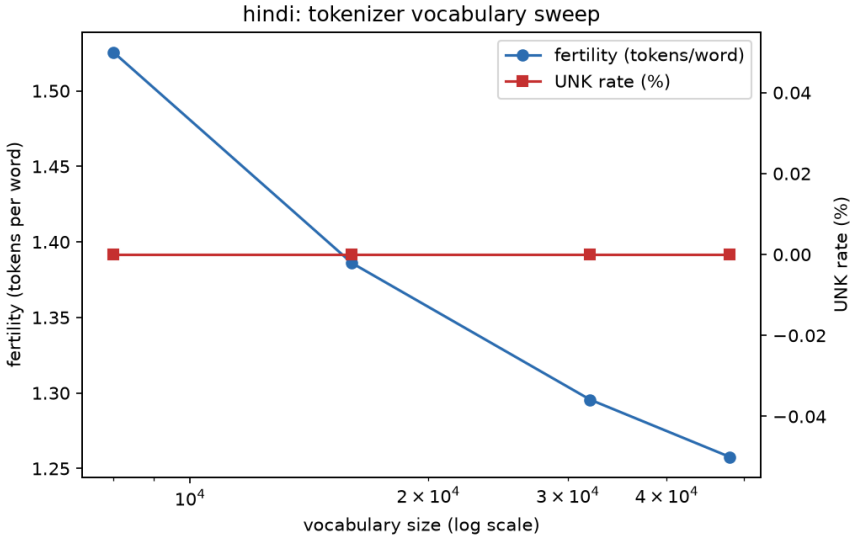



Figure 1: Hindi Vocab Sweep: How vocabulary size affects fertility rate and chars-per-token.

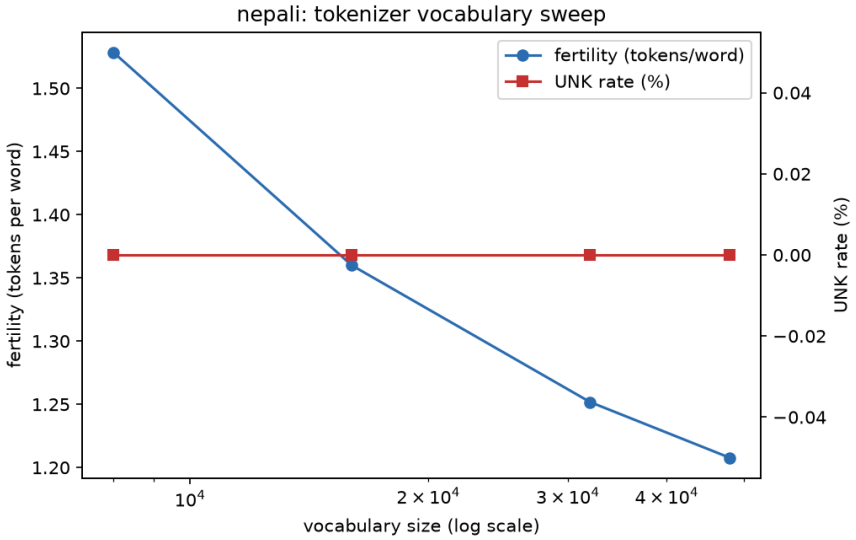



Figure 2: Nepali Vocab Sweep: How vocabulary size affects fertility rate and chars-per-token.

# 3 Phase 2: Transformer Model

## 3.1 Architecture

I wrote a decoder-only Transformer model in PyTorch from scratch (no HuggingFace classes). To fit the 25M parameter budget, I went with: - **Size:** $d_{model} = 384$, 10 layers, 6 attention heads. Total params: 24,070,656. - **Embeddings:** Learned absolute positional embeddings (no RoPE). I tied the input and output embeddings to save about 6 million parameters, which I spent on making the model deeper. - **FFN:** Standard GELU activation with $d_{ff} = 4 \times d_{model}$. - **Norm:** Pre-norm so gradients flow better.

## 3.2 Hyperparameter Tuning

I trained both models using AdamW ($\beta = (0.9, 0.95)$, weight decay 0.1), fp16 autocast with a GradScaler, on Kaggle T4 GPUs. The batch was 8 sequences $\times$ 16 gradient accumulation steps =

128 sequences = **65,536 tokens per step**.

Instead of guessing the learning rate, I ran a sweep on Hindi for 1,500 steps. I found that removing dropout entirely (0.0 instead of 0.1) and bumping the peak learning rate up to 3e-3 (from 6e-4) improved validation loss significantly. The four configs tested were:

Table 3: Hindi LR Sweep (1,500 steps, all else equal)

| Dropout | Peak LR | Val loss @1500 | vs baseline |
|---|---|---|---|
| 0.1 | 6e-4 | 4.1865 | baseline |
| 0.0 | 6e-4 | 4.1167 | −0.070 |
| 0.0 | 1.5e-3 | 3.9924 | −0.194 |
| **0.0** | **3e-3** | **3.9492** | **−0.237** |
| 0.0 | 6e-3 | 3.9729 | −0.214 |

The optimum is interior (3e-3 beats both 1.5e-3 and 6e-3), which means it is bracketed and not just the edge of my grid. Carried to the full 8,732-step run, the combined change gave a **16.2% perplexity reduction** (35.57 to 29.81).

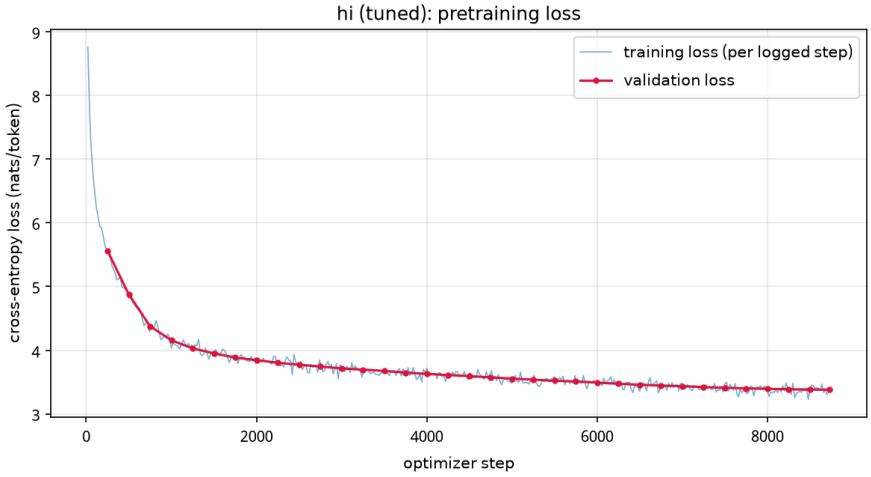



Figure 3: Hindi Training & Validation Loss over 1 full epoch.

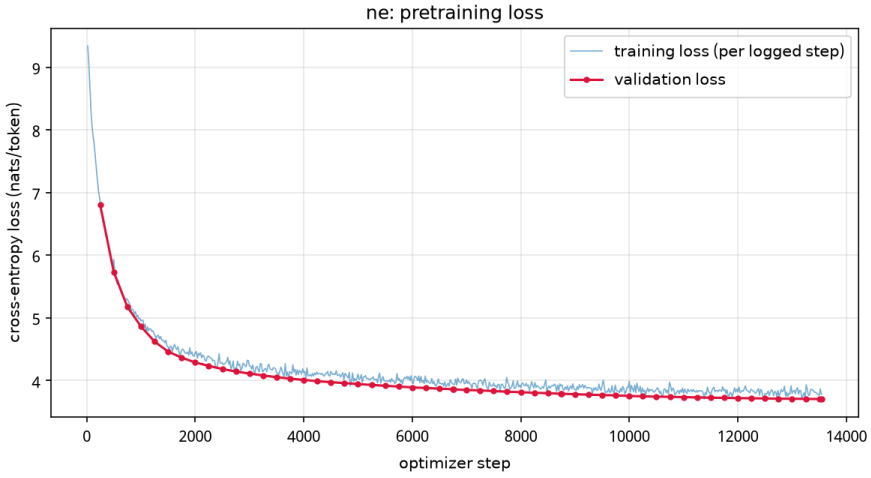



Figure 4: Nepali Training & Validation Loss over 1 full epoch.

### 3.3 Evaluation: Why BPB is better than Perplexity

When I scored the models on the held-out test set, the results looked weird at first glance.

Table 4: Phase 2 Test Metrics

| Metric | Hindi (H) | Nepali (L) |
|---|---|---|
| Test Perplexity | **29.81** | 40.70 |
| Test Bits-Per-Byte (BPB) | 0.4947 | **0.4314** |

Hindi has a much better (lower) perplexity, but Nepali has a better (lower) Bits-Per-Byte. This is exactly why we can't compare perplexity across different tokenizers! Nepali's tokenizer packs more characters into a single token. More text per token means fewer, harder predictions, which makes perplexity look worse. But when you normalize it by bytes (BPB), Nepali is actually cheaper to predict. This is mainly because the Nepali dataset has more boilerplate and near-duplicates, making it an "easier" corpus, not necessarily a smarter model.

### 3.4 Generation Quality

I also evaluated the models by having them continue held-out test passages (first 64 tokens as prompt, generate up to 128 tokens), scoring with BLEU-4, chrF, and ROUGE-L. Honestly, the results confirmed what many papers say: n-gram overlap metrics are weak for open-ended generation in morphologically rich languages like Hindi and Nepali.

Table 5: Generation Metrics (Greedy Decoding vs Temperature 0.5)

| Model | Decoding | BLEU-4 | chrF | ROUGE-L | Rep-4 (↓) | Distinct-2 (↑) |
|---|---|---|---|---|---|---|
| Hindi (H) | Greedy | 3.12 | 15.53 | 0.1124 | 0.578 | 0.176 |
| Hindi (H) | Temp 0.5 | **3.53** | **18.56** | **0.1252** | 0.244 | 0.419 |
| Nepali (L) | Greedy | 1.55 | 14.42 | 0.0916 | 0.586 | 0.161 |
| Nepali (L) | Temp 0.5 | **2.30** | **17.97** | **0.1112** | 0.235 | 0.452 |

The most interesting thing from these results was not the overlap scores themselves, but the repetition numbers. Greedy decoding repeats 57–59% of its 4-grams (Rep-4 ˜0.58 for both), which matches what Holtzman et al. found with large models. Sampling at temperature 0.5 drops that to 24%, which is much more natural. Also, BLEU-4 and chrF actually extitdisagreed on which decoding setting is best: BLEU peaked at temperature 0.5 while chrF peaked at temperature 1.0. I checked and the reason is that greedy output partly "continues" the prompt by paraphrasing it back, which tricks BLEU into a higher score. The diversity metrics (Distinct-2) are the honest signal here.

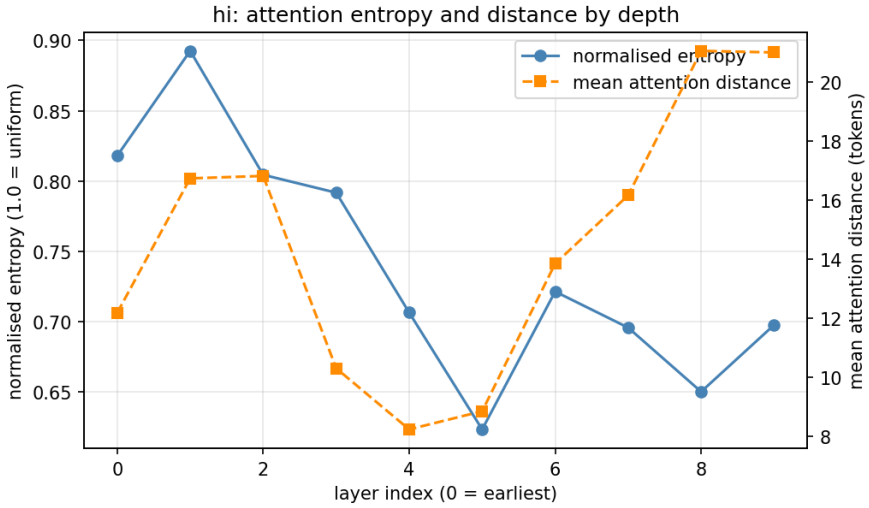



Figure 5: Hindi Attention Layer Profile: The model learned to place a sharp "previous-token" head at exactly Layer 5, Head 1.

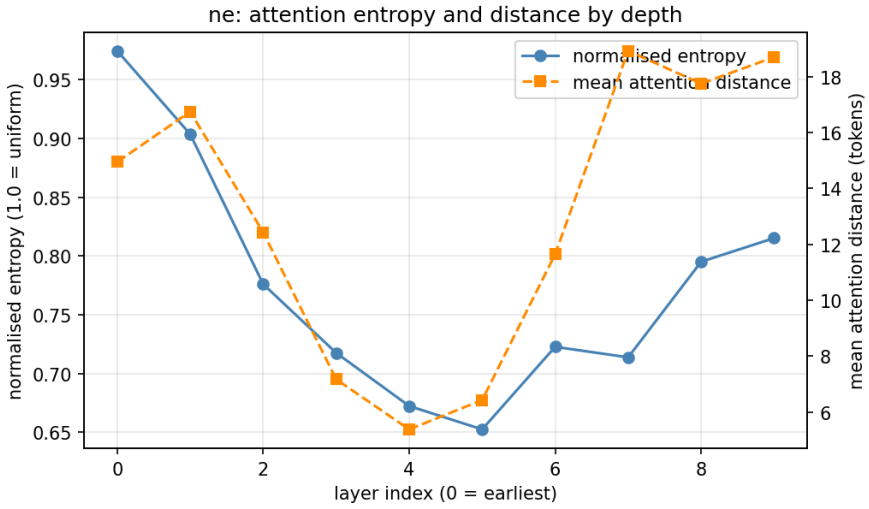



Figure 6: Nepali Attention Layer Profile: The model learned to place a sharp "previous-token" head at exactly Layer 5, Head 1.

### 3.5 Bonus: Ablating Positional Embeddings

As an extra experiment, I completely removed the positional embeddings from the Nepali model (bringing it down to 23,874,048 parameters) and trained it from scratch. You might expect a Transformer to completely fail without positional embeddings since self-attention is order-blind.

Surprisingly, it only cost **+0.0255 nats** in cross-entropy loss (where "nats" is the natural-logarithm unit of loss) and resulted in a 2.6% worse perplexity (41.75 vs 40.70). To put that in perspective, a penalty of 0.0255 nats is extremely small (less than what the model learns from just a few thousand steps of training data). By analyzing the attention matrices, I found that the model reconstructed positional information using the causal mask. It did this by making its layers 1 and 2 pay near-uniform attention across the visible prefix. Since the amount of visible context grows with each step, averaging over it gives the model a proxy for its current position. This proves that at short context lengths (512 tokens), explicit positional embeddings are helpful but almost entirely redundant.

# 4 Phase 3: Reasoning Tasks

## 4.1 Task Setup

In Phase 3, I wanted to see if these models could do comparative and transitive reasoning. I generated synthetic datasets for both languages with these tasks:

1. **P1 (Comparison over two):** E.g., "Who is taller?"
2. **P2 (Multi-hop chaining):** $A > B$, $B > C \Rightarrow A$ vs $C$.
3. **P3 (Superlative over three):** "Which is smallest?"
4. **P4 (Ordinal over four):** "Second tallest?"
5. **P6 (4-hop chain):** Out-of-distribution (OOD) test. The model was never trained on chains this long.

To stop the model from cheating, I used three structural holdouts, not just an n-gram filter:

- **Entity holdout:** Train uses 69 entity names, val/test use 15 completely different names with zero overlap.
- **Relation holdout:** 2 relation words (*speed* and *wealth*) never appear in training at all.
- **Hop holdout:** Training goes up to 3 hops; the OOD split tests 4-hop chains the model has never seen.

The dataset has 60,000 training examples, 6,000 val, 6,000 test, and 2,000 OOD examples. Each example is on average 23.3 tokens long (longest is 35), nothing was truncated, and only 17.2% of tokens actually carry loss (the prompt is masked out, so the model is only scored on its answer).

## 4.2 Results

I fine-tuned the models for 15 epochs (3,525 total steps, batch size 256, context window reduced from 512 to 128 since the longest reasoning example is only 35 tokens). A major bug I had to fix during this phase was left-padding in my batched decoder, which corrupted the position embeddings (since the models use learned absolute embeddings, shifting a token to a wrong position gives it a completely wrong embedding). The bug showed up as accuracy depending on batch size: batch-of-1 gave 1.000, batch-of-60 gave 0.517. After fixing it with right-padding and a per-row gather of each sequence's last token index, and re-tuning the learning rate to 3e-05 for both languages, the results were solid.

Table 6: Phase 3 Exact Match Accuracy

| Task Type | Hindi Pretrained | Hindi Finetuned | Nepali Pretrained | Nepali Finetuned |
|---|---|---|---|---|
| P1: Comparison (2 entities) | 0.494 | **0.979** | 0.501 | **0.876** |
| P2: Multi-hop (3 entities) | 0.517 | **0.963** | 0.518 | **0.892** |
| P3: Superlative (3 entities) | 0.260 | **0.957** | 0.198 | **0.868** |
| P4: Ordinal (4 entities) | 0.379 | **0.918** | 0.369 | **0.824** |
| **Overall Exact Match** | (free gen is 0.0) | **0.954** | (free gen is 0.0) | **0.835** |
| **OOD 4-hop chain (Length Generalization)** | 0.090 | **0.023** | 0.097 | **0.140** |

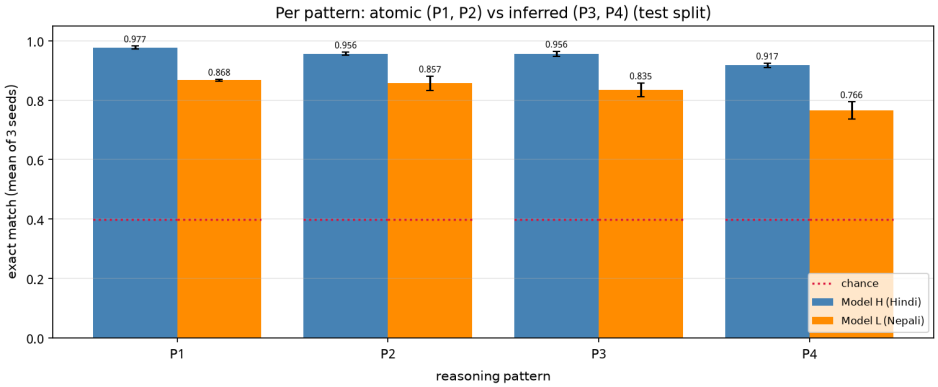



Figure 7: Error rate breakdown for Hindi across different tasks and variables.

The models learned the tasks well, jumping from random chance to 83-95% accuracy. However, they completely failed the OOD 4-hop test. When I looked closely, I found that on the OOD set, Hindi answered with a "middle" entity 98.3% of the time, even though the correct answer is always an extreme. This proves it learned a positional shortcut ("pick the 3rd name") instead of real logic.

### 4.3 Bonus: Chain-of-Thought (CoT) vs Answer-Only

I also ran an A/B test comparing the standard "Answer-Only" format against generating a full reasoning trace before answering (Chain-of-Thought). For example, instead of just outputting *"Answer: Rajendra"*, the CoT model is trained to output *"Rajendra is lighter than Sharma, and Sharma is lighter than Sachin... Answer: Rajendra"*. The results showed a fascinating trade-off:

- **In-domain (Test Exact Match):** This tests the model on the same 1-to-3 hop chain lengths it saw during training. Here, Answer-only was strictly better (96.0% accuracy vs CoT's 86.5%).
- **Out-of-domain (4-hop OOD):** This tests if the model can generalize to longer 4-hop chains that it was **never trained on**. Here, CoT was **almost 5× better** (9.7% accuracy vs Answer-only's 2.0%).

What does this mean? Even though 9.7% is still a poor accuracy score (random chance is 20% for a 5-entity chain), it is a massive relative improvement over 2.0%. It shows that the Answer-only model is just memorizing positional shortcuts (e.g., "always pick the 3rd name I see"). By forcing the CoT model to explicitly write out the intermediate comparisons step-by-step, we break its reliance on those cheap positional shortcuts, helping it generalize slightly better to longer chains.

### 4.4 Attention Mechanism

By probing the attention weights, I found that both models (despite sharing no data or weights) built the exact same mechanism to solve the task. At the final answer token, the strongest attention head for retrieving the right entity was **Layer 9, Head 4 (L9H4)** in both Hindi and Nepali.

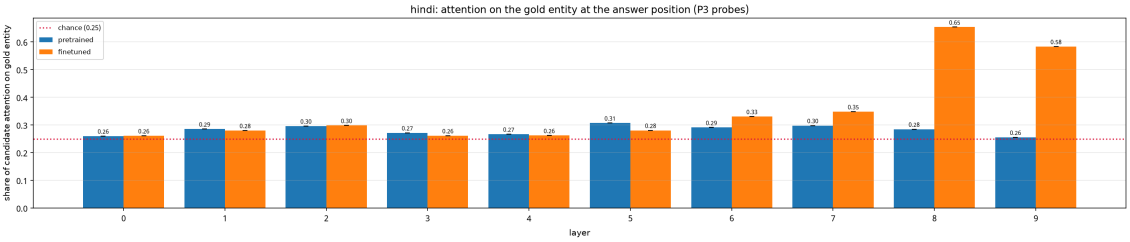



Figure 8: Hindi Attention Gold Share: Notice how attention on the correct target token shoots up in the late layers (L9) after finetuning.

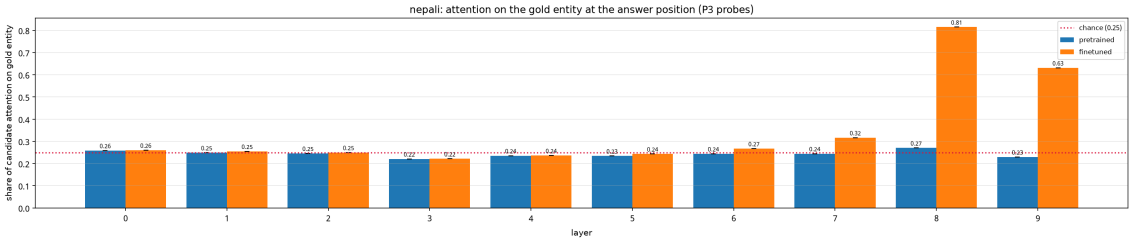



Figure 9: Nepali Attention Gold Share: Notice how attention on the correct target token shoots up in the late layers (L9) after finetuning.

## 5 Answers to Questions

**1. How did data scale and quality differ between the two languages?** Even though Nepali is supposed to be "lower-resource", I actually collected more tokens for it (903M vs Hindi's 582M). The big difference was quality and diversity. My Nepali data came mostly from news sites that syndicate each other's articles, so its near-duplicate rate was 4x higher than Hindi's (5.24% vs 1.27%).

**2. How do language-modeling and reasoning results compare across tiers?** They go in opposite directions! Because Nepali's corpus was more repetitive and its tokenizer packed more characters per token, it got a better Bits-Per-Byte score (0.4314 vs 0.4947). But for reasoning, Hindi crushed Nepali (95.4% vs 83.5% exact match). Being easier to predict per-byte didn't make the Nepali model a smarter reasoner.

**3. Which tokenizer or corpus factors most hurt Model L (Nepali)?** Surprisingly, it wasn't the tokenizer. Nepali's tokenizer was actually more efficient than Hindi's (4.644 chars/token vs 3.892). The main culprit I found was **in-context copying**. I ran what's called an "induction probe": I gave the pretrained model a sequence of random tokens like `[A, B, C, A, B, ?]` and checked whether it could predict `C` (i.e., copy the pattern). This tests whether the model has learned to look back in context and repeat, which is exactly the skill needed for transitive reasoning (you have to copy an entity name out of the premise and write it as the answer). Hindi scored **0.680**, while Nepali scored only **0.364**, roughly 2× worse. My best guess for why is Nepali's highly repetitive corpus: when your training data is full of near-identical news articles, you don't need to learn to copy from context because the next token is always predictable from local patterns anyway.

**4. What evidence explains the differences?** The biggest thing I learned here: most of the apparent gap was actually a **bug in my own code**. I had been training Nepali at LR 3e-06 instead of 3e-05, which badly undertrained it. After fixing that, Nepali's accuracy jumped from 0.525 to 0.835, closing about 72% of what had looked like a 43-point language gap. So the first and most important piece of evidence is: equalise your recipe before drawing conclusions about language.

After fixing it, the cleanest evidence for a real remaining difference is the **held-out relation transfer test**. Both models did well on the 7 relation words they trained on (height, weight, age, etc.). But when I tested on 2 completely unseen relation words (speed and wealth, which never appeared anywhere in training):

Table 7: Transfer to Held-Out Relation Words (Exact Match)

| Model | Seen relations | Held-out relations | Drop |
|---|---|---|---|
| Hindi (H) | 0.983 | 0.848 | −13.5 pts |
| Nepali (L) | 0.923 | 0.518 | −40.5 pts |

Both still beat the 0.398 random chance baseline, so they both transfer *something*. But Nepali loses 3× as much as Hindi when the relation word is new. This connects directly back to Q3: using an unseen relation word forces the model to rely on the *structure* of the prompt (read the premises, find the right entity, copy it out) rather than just pattern-matching on a familiar word. That structural reliance on copying from context is exactly what the induction probe showed Nepali is weak at. Neither model truly "learned to reason" in a general sense, because both failed the 4-hop OOD test by falling back on positional shortcuts instead.

## 6 Conclusion

In this project, I built everything from tokenizers to Transformers from scratch for Hindi and Nepali. I learned that you can't compare perplexities across tokenizers, BPB is the only fair metric, and a more repetitive corpus gives a great BPB but hurts reasoning transfer. Finally, while small models can easily learn to copy answers for specific patterns (using heads like L9H4), they are mostly finding positional shortcuts rather than doing true symbolic logic.